PYTHON INTERNALS

Анатомія пам’яті

Купа, GC та Алокатори

Мистецтво ефективного використання ресурсів: від структури об'єктів до механізмів збирання сміття та оптимізації.

Сумець Світлана

2026

Пам'ять

Що таке пам'ять у Python?

> У Python пам'ять - це приватна динамічна купа (Private Heap), де зберігаються всі об'єкти та структури даних. Ви не керуєте нею напряму - це робота Memory Manager.

Динамічна купа (Heap)

Основне сховище для всіх об'єктів Python. Розмір купи змінюється динамічно в залежності від потреб програми під час виконання.

Менеджер пам'яті (Memory Manager)

Компонент інтерпретатора, що автоматично виділяє пам'ять для нових об'єктів та звільняє її, коли вони більше не потрібні.

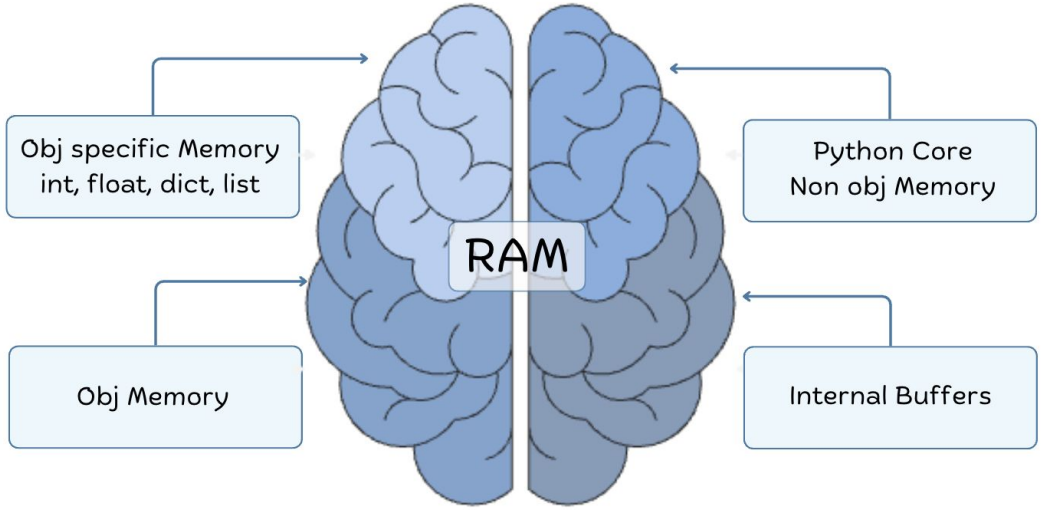
Категорії пам'яті

Пам'ять чітко розділена: об'єктна (для даних Python) та необ'єктна (стек викликів, буфери, внутрішні структури C).

Вплив на продуктивність

Ефективне управління пам'яттю критичне для швидкодії. Часте виділення/звільнення (allocation/deallocation) може сповільнити програму.

СТРУКТУРА ПАМ'ЯТІ



Важливо: Вся пам'ять Python є приватною. ОС виділяє її процесу Python, і інші процеси не мають до неї прямого доступу.

Типи пам'яті в Python

OPTIMIZED

Об’єктно-специфічна пам’ять

Пам’ять для об’єктів зі спеціалізованими механізмами управління (list, dict). Використовує власні алокатори для швидкодії.

- Спеціальні алокатори: для list, dict, array
- Внутрішні структури: дерева, графи
- Пули пам'яті: попереднє резервування
- Мета: швидке виділення/звільнення

MAIN STORAGE

Пам’ять об’єктів

Основне сховище для всіх об'єктів Python (числа, рядки, класи). Зберігає як дані, так і службові метадані.

- Payload: реальні дані (int, str)
- Метадані: ob_type, стан
- Refcount: ob_refcnt для GC
- PyObject: базовий заголовок

KERNEL

Необ’єктна пам’ять ядра

Службова пам'ять для роботи самого інтерпретатора. Дані, що не є об'єктами Python, але критичні для виконання.

- Стек викликів: Call Stack
- Символи: таблиці імен, scope
- Потоки: thread state info
- Config: налаштування середовища

TEMPORARY

Внутрішні буфери

Області для тимчасового зберігання даних під час операцій. Оптимізують взаємодію з ОС та компіляцію.

- I/O: буфери файлів та мережі
- Bytecode: буфери компіляції
- Кеші: гарячі дані, results
- Мета: зменшення системних викликів

Ієрархія алокаторів CPython

Specific Allocators	`Для list, dict, context`	HIGH LEVEL API	Знають структуру
Object Allocator	`pymalloc`	SMALL OBJECTS	Швидкий шлях
Raw Memory Allocator	`PyMem_RawMalloc`	LARGE OBJECTS	Загальний інтерфейс
Universal Allocator	`malloc/free (C Library)`	OS LEVEL	Системні виклики

Threshold: 512 bytes

Абстракція

Python приховує керування пам'яттю за кількома шарами абстракції. Це дозволяє змінювати стратегії (наприклад, використовувати pymalloc) без зміни коду C-розширень.

pymalloc

Спеціалізований алокатор для малих об'єктів (< 512 байт). Мінімізує кількість системних викликів malloc і зменшує фрагментацію пам'яті.

Великі об'єкти

Об'єкти більші за 512 байт передаються безпосередньо системному алокатору (malloc), оскільки для них накладні витрати на керування пулами не є виправданими.

Фрагментація пам’яті

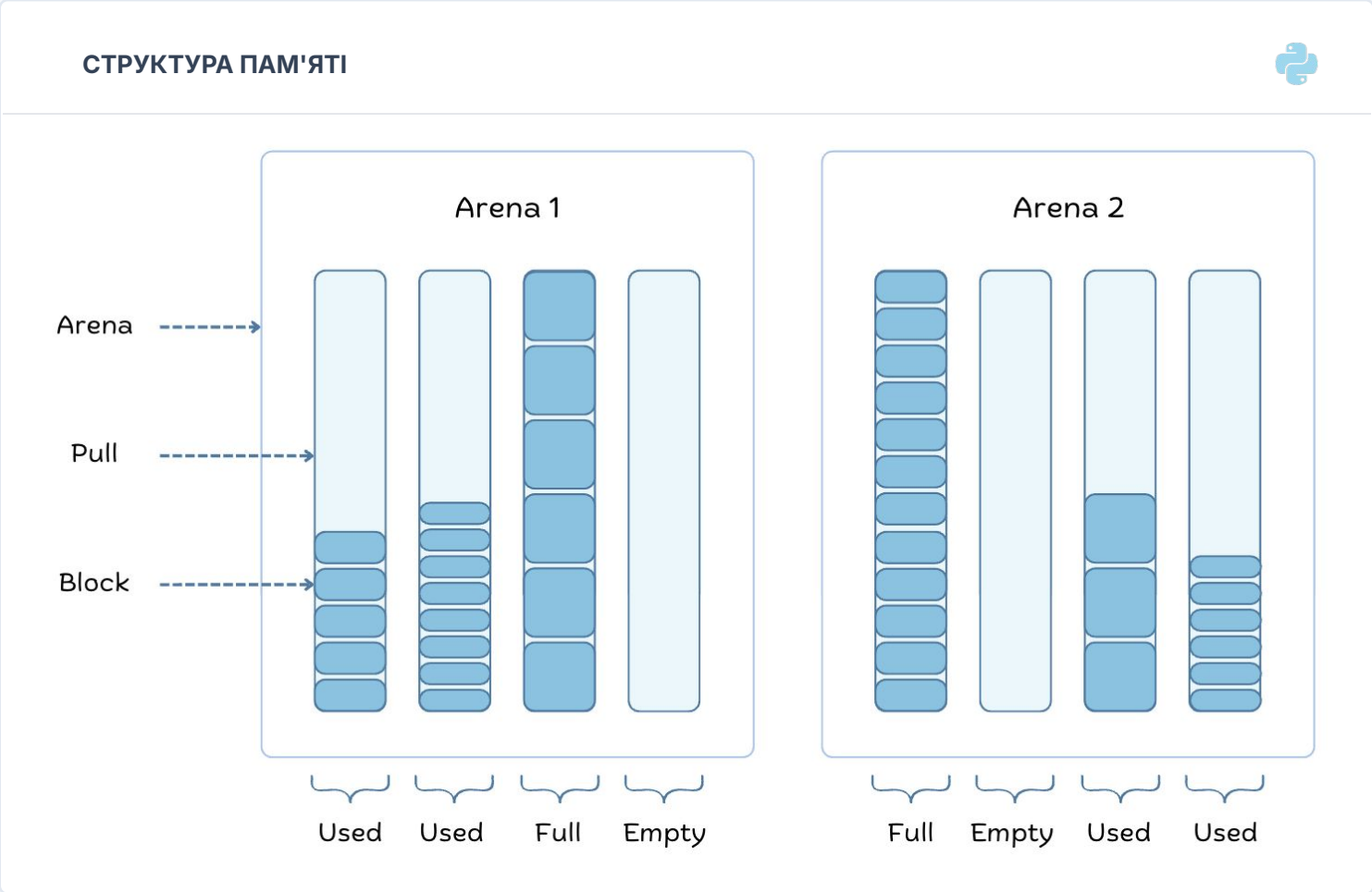
Фрагментація

Фрагментація пам’яті - це проблема, коли вільне місце в пам’яті розбивається на дрібні шматочки, і стає важко знайти достатньо великого безперервного простору для нових даних.

Приклад

Якби ми мали багато маленьких вільних місць у валізі, але не могли б покласти туди великий предмет, бо всі ці місця розкидані й не з’єднані в єдиний простір.

Арени, Пули та Блоки



Arena 256 KB

Арени виділяються безпосередньо від ОС через mmap(). Містять 64 пули.

Pool (e.g. Size Class 32B) 4 KB

Пул обслуговує об'єкти фіксованого розміру. Мінімізує фрагментацію та malloc/free.

Block FIX SIZE (e.g. Size 32 B)

Найменша одиниця алокації.
Має фіксований розмір.
Використовується для одного об'єкта.

Алокація малого об'єкта: покроково

1. Запит

```
obj = ClassA()
# C Level:
PyObject_Malloc(28)
```

Користувач створює об'єкт. Інтерпретатор запитує N байт.

2. Клас розміру

Request: 28 bytes

Align 8 bytes

↓

Class 32 Bytes

Вирівнювання до найближчого кратного 8 байт.

3. Вибір пулу

- Pool A (Size 16B)
- **Pool B (Size 32B)**
- Pool C (Size 32B) FULL
- Pool D (Size 64B)

Пошук першого пулу відповідного класу з вільними блоками.

4. Free List

Pool state: free list head

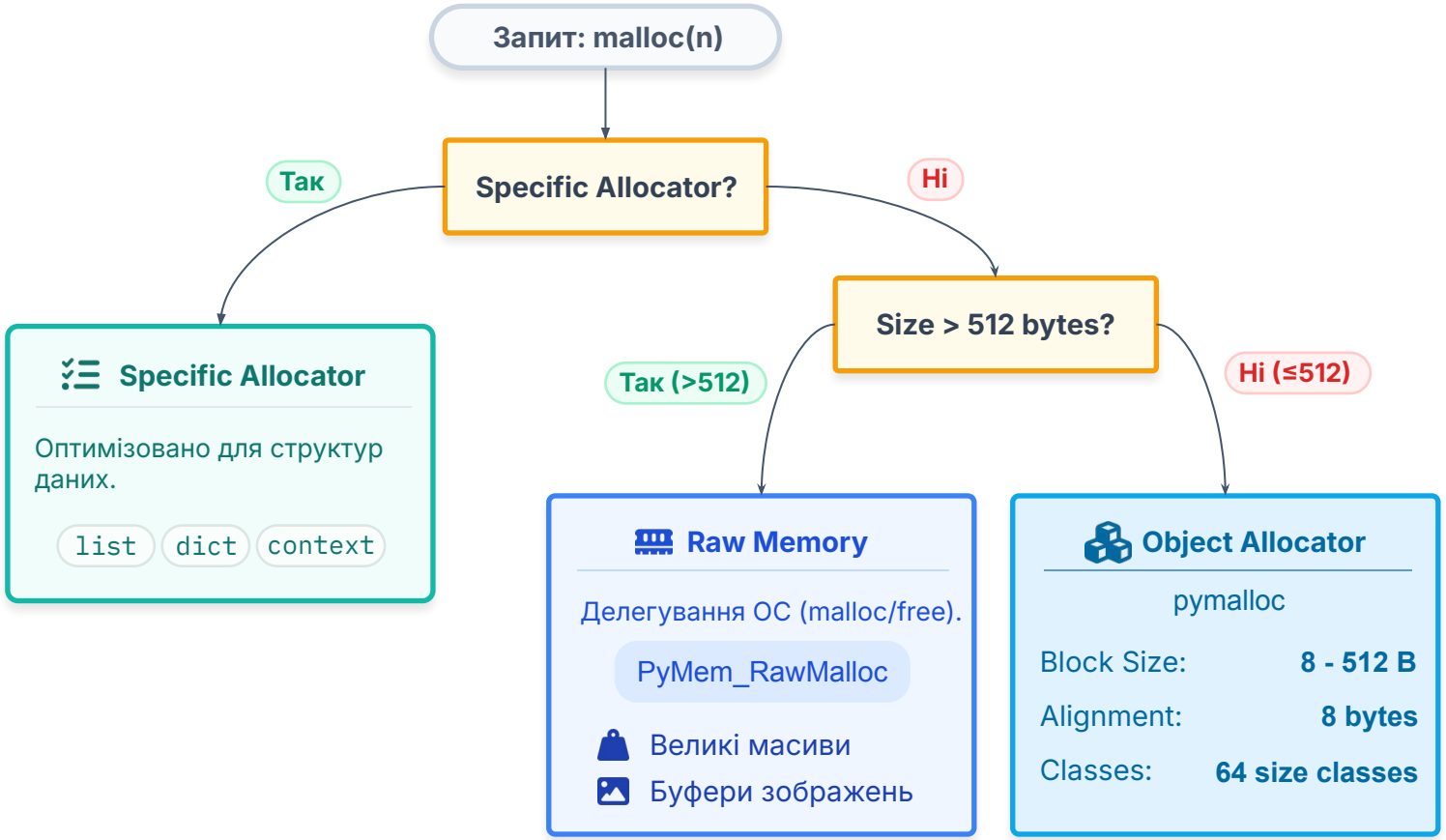
freeblock_ptr

Allocated Block | Block B (Next) ptr → C

Видає адресу першого вільного блоку. Оновлює вказівник на наступний.

Якщо всі пули переповнені, створюється нова арена.

Логіка вибору алокатора



РОЗПОДІЛ ОБ'ЄКТІВ

~93.5%

Об'єктів менші за 512 байт і проходять через pymalloc.

ШВИДКІСТЬ (SMALL OBJ)

High Performance

Використання free lists дозволяє алокувати пам'ять за O(1) без системних викликів.

SIZE CLASSES

Idx 0	Idx 1	...	Idx 63
8B	16B	...	512B

Кожен клас має власний пул блоків фіксованого розміру.

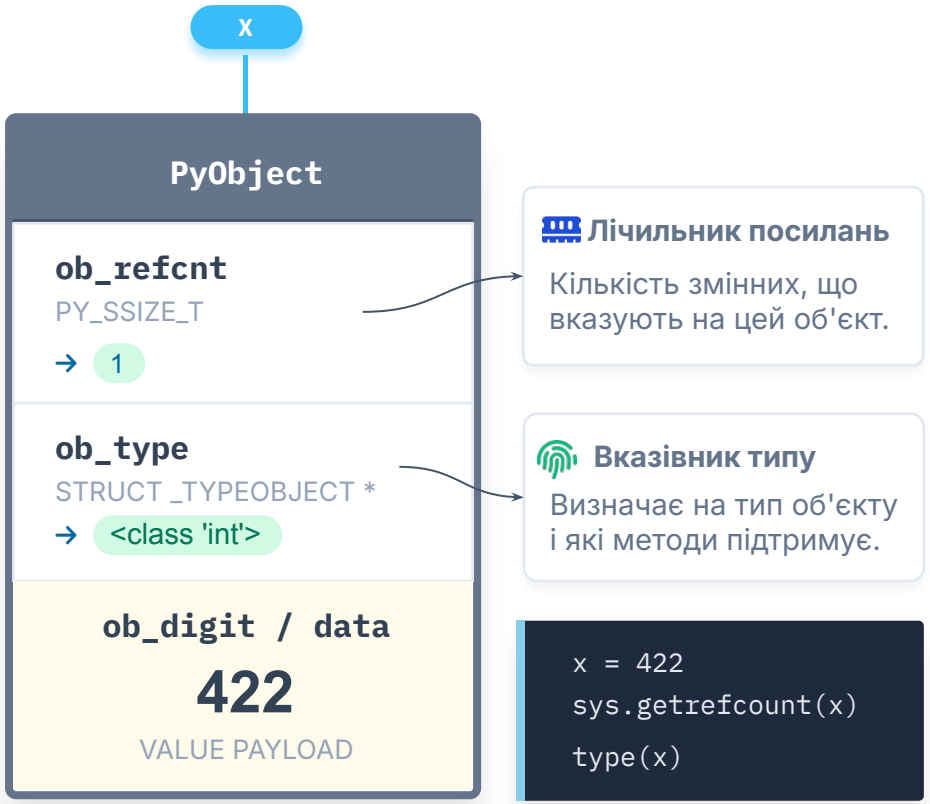
ПОРІГ ПЕРЕМИКАННЯ

512 Bytes

Все, що більше, обробляється як "raw memory" для уникнення засмічення пулів.

Ключовий принцип: Python оптимізує роботу з пам'яттю, розділяючи об'єкти за розміром. pymalloc обробляє дрібні об'єкти (до 512 байт), використовуючи size classes (8, 16, 24...512) для мінімізації фрагментації. Це покриває ~90-95% всіх об'єктів у типовій програмі.

Анатомія об'єкта Python



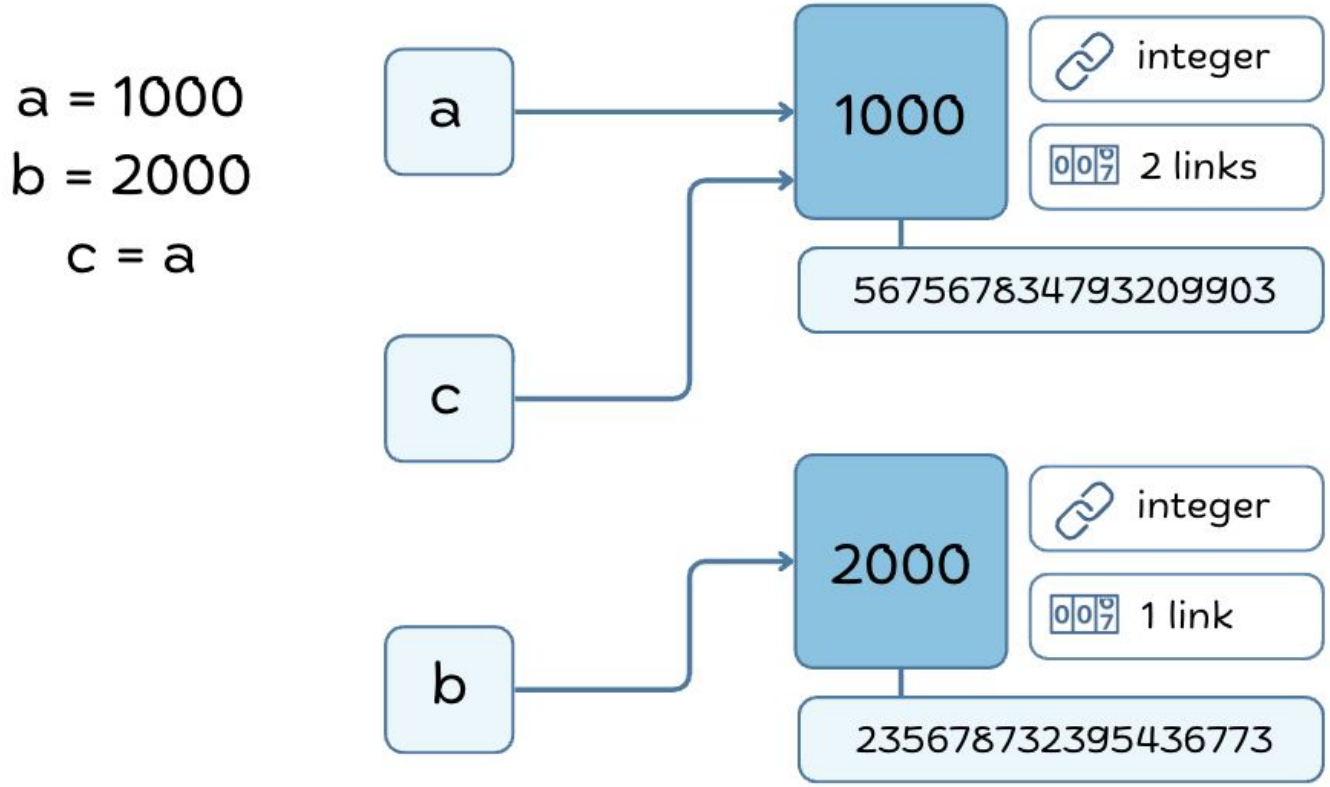
Лічильник посилань

Кількість змінних, що вказують на цей об'єкт.

Вказівник типу

Визначає на тип об'єкту і які методи підтримує.

```
x = 422
sys.getrefcount(x)
type(x)
```



Все є об'єктом

Навіть прості числа в Python мають ці заголовки, тому `int` займає більше 4 байт.

Поліморфізм

Завдяки `ob_type`, Python знає, як поводитись з об'єктом у runtime.

Overhead

Мінімальний розмір об'єкта ~28 байт (на 64-бітній системі) через метадані.

Життєвий цикл: Reference Counting

+ Allocation

СТВОРЕННЯ

1

`x = Object()`

Об'єкт створено, перше посилання присвоєно змінній x.

New reference

ПРИСВОЄННЯ

2

`y = x`

Створено нове посилання y. Лічильник збільшується (Py_INCREF).

Del Reference

ВИДАЛЕННЯ

1

`del y`

Посилання y видалено. Лічильник зменшується (Py_DECREF).

Deallocation

ЗВІЛЬНЕННЯ

0

`del x`

Лічильник досяг нуля. Пам'ять повертається в пул негайно.

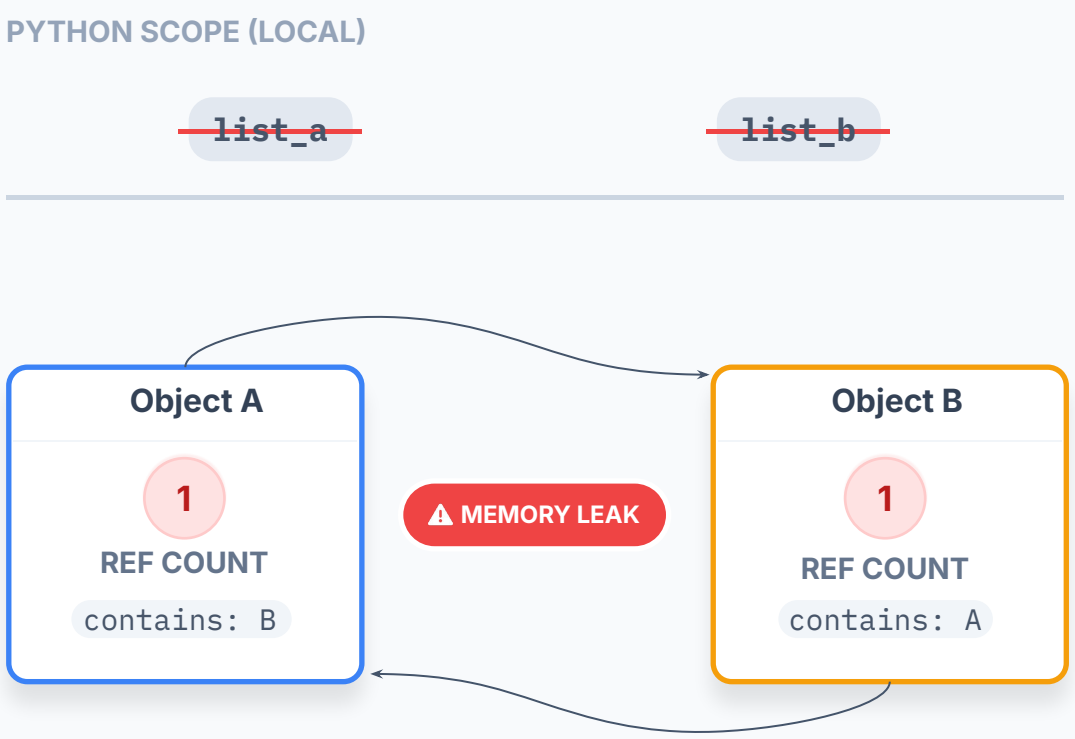
Миттєва дія

Головна перевага Reference Counting: об'єкт знищується одразу, як тільки стає непотрібним. Немає пауз "Stop-the-world".

Критерій існування

Лічильник - єдиний критерій життя об’єкта.Якщо refcount > 0, об’єкт існує, незалежно від його логічної корисності. Reference Counting працює з посиланнями, а не з семантикою.

Проблема циклічних посилань



Об'єкти недосяжні з програми, але посилаються один на одного

cycle.py

```python
a = []
b = []
a.append(b) # a -> b
b.append(a) # b -> a

del a
del b

# Результат: ref_cnt = 1
# Reference Counting не видалить це!
```

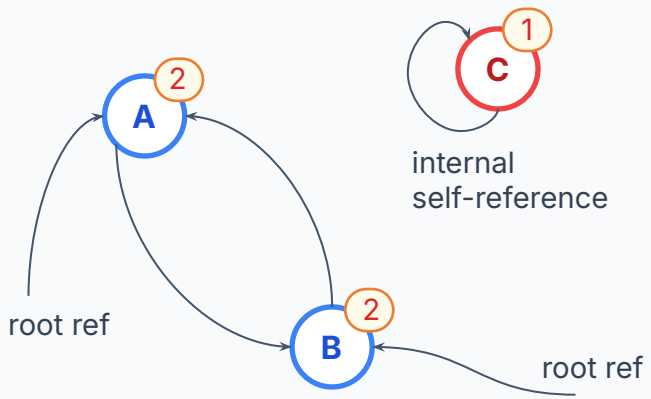
Пастка Reference Counting

Механізм підрахунку посилань працює миттєво, але він сліпий до циклів. Оскільки лічильник не дорівнює нулю (кожен об'єкт тримає іншого), пам'ять ніколи не звільниться автоматично.

→ **Рішення: Garbage Collector**

Алгоритм пошуку циклів

1. СКАНУВАННЯ



GC перебирає всі контейнери (list, dict, class) у поколінні.

2. GC_REFS--

A	refcnt: 2 →	gc_refs: 1
B	refcnt: 2 →	gc_refs: 1
C	refcnt: 1 →	gc_refs: 0

gc_refs = refcnt – internal_references

Створюється копія лічильників (gc_refs). Віднімаються посилання всередині списку.

3. ІЗОЛЯЦІЯ

A	gc_refs > 0	REACHABLE
B	gc_refs > 0	REACHABLE
C	gc_refs == 0	UNREACHABLE

Об'єкти з gc_refs == 0 ізолюються. Якщо вони не досяжні ззовні - це сміття (Cycle Garbage).

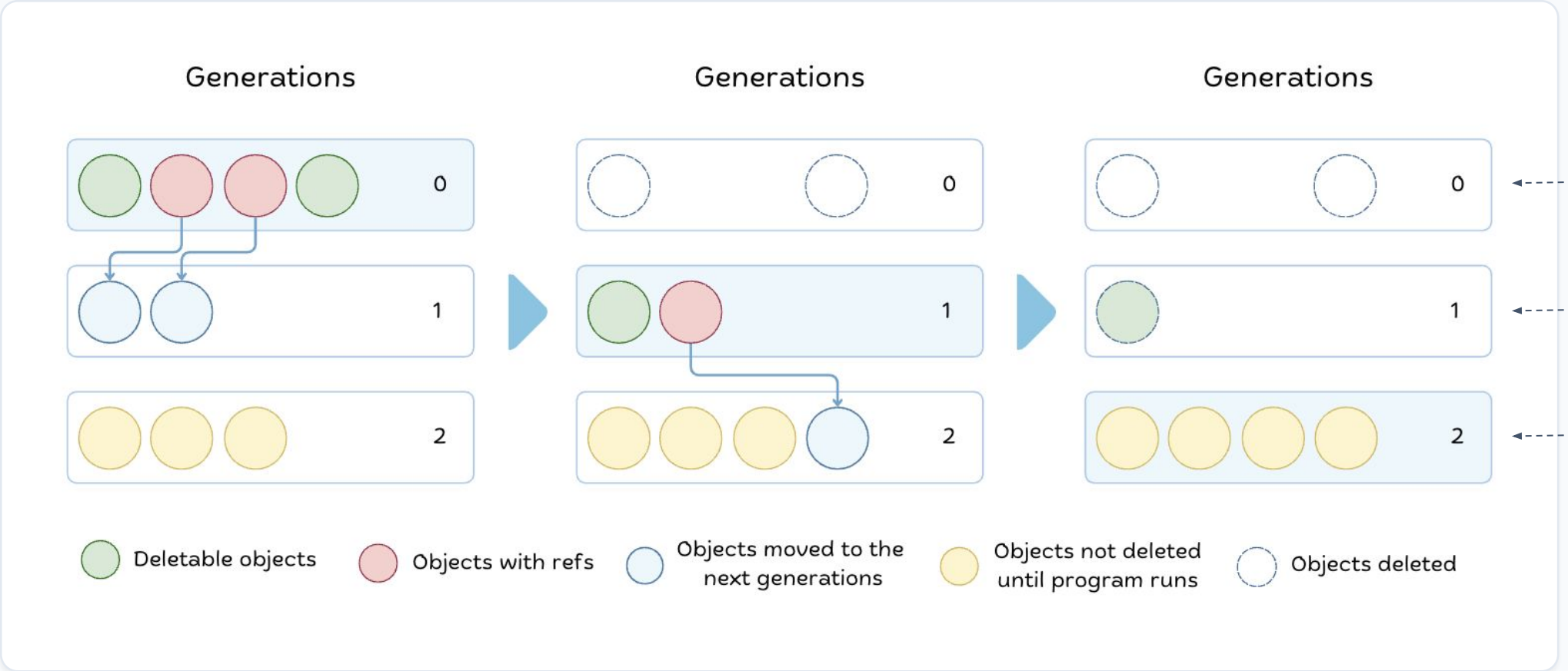
Ключова ідея

GC не видаляє об'єкти відразу. Він знаходить ізольовані групи об'єктів (цикли), які посилаються тільки один на одного, і переміщує їх до списку Unreachable для подальшого видалення.

OVERHEAD

Stop-the-World

Система трьох поколінь GC



Frequency:

Дуже часто
Кожні 700 алокацій

Рідше
Кожні 10 зборів Gen0

Рідко
Кожні 10 зборів Gen1

Пороги та тригери GC

Стандартні пороги (Thresholds)

Значення за замовчуванням у CPython

GEN 0 — ALLOCATION COUNT

700

Різниця між створеними та видаленими об'єктами.

GEN 1 — COLLECTION COUNT

10

Кількість зборів Gen0 перед запуску Gen1.

GEN 2 — COLLECTION COUNT

10

Кількість зборів Gen1 перед повним (Full) збором.

```
gc.get_threshold()  # (700, 10, 10)
```

Логіка спрацювання

Як алгоритм ухвалює рішення про старт

`allocs - deallocs > threshold0`

Збір запускається тільки для покоління 0, коли чистий приріст об'єктів перевищує 700. Це запобігає частим запускам при активному створенні/видаленні тимчасових змінних.

Ручний контроль

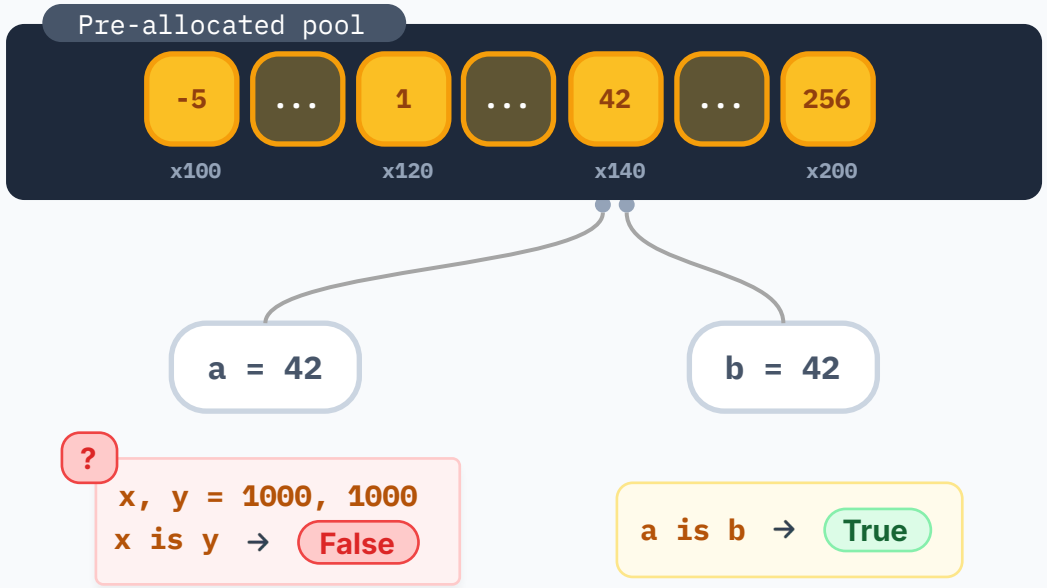
Можна примусово запустити повний збір сміття (всі покоління), що корисно для серверів перед переходом в режим очікування.

```
n = gc.collect()      # Returns unreachable obj count
```

Інтернування: Повторне використання

Малі цілі числа

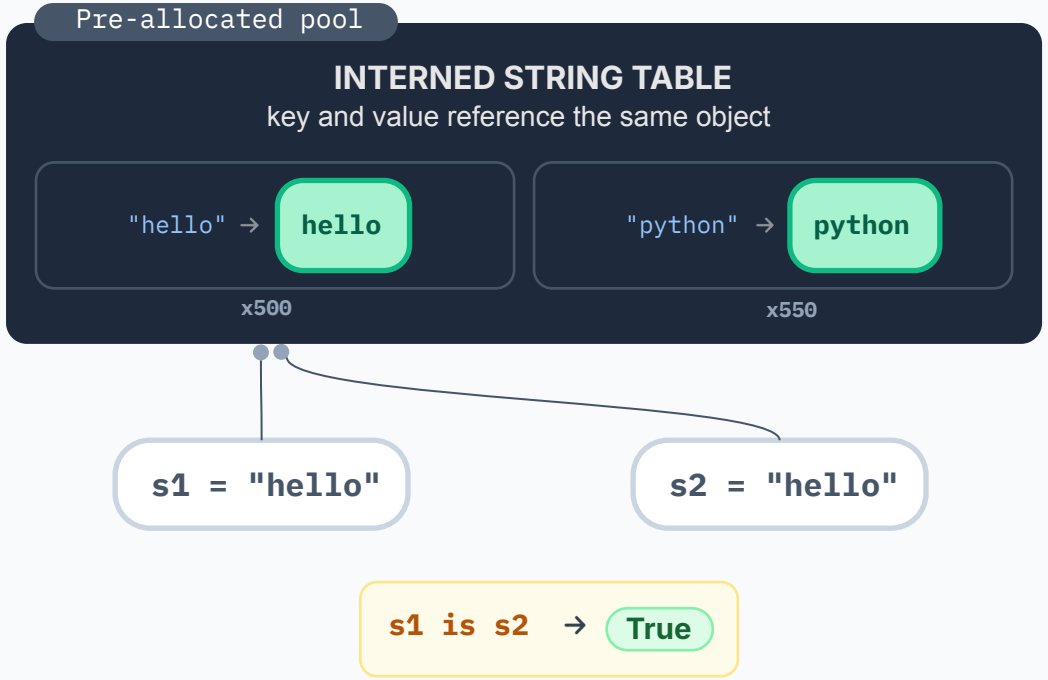
Range: -5 .. 256



Ці об'єкти створюються автоматично при старті інтерпретатора. Будь-яка змінна з цим значенням посилається на одну й ту саму адресу в пам'яті.

Рядки (String Interning)

sys.intern()



Ідентифікатори (імена змінних) та короткі рядки інтернуються неявно. Це економить пам'ять та прискорює пошук у словниках.

Динамічне резервування (Over-allocation)

Стратегія List Append

0 1 2 3 ? ? ? ?

SIZE: 4 **CAPACITY: 8**

Python не розширює список на 1 елемент. Він виділяє пам'ять із запасом (наприклад, 4 → 8 → 16), щоб уникнути частого копіювання масиву (realloc).

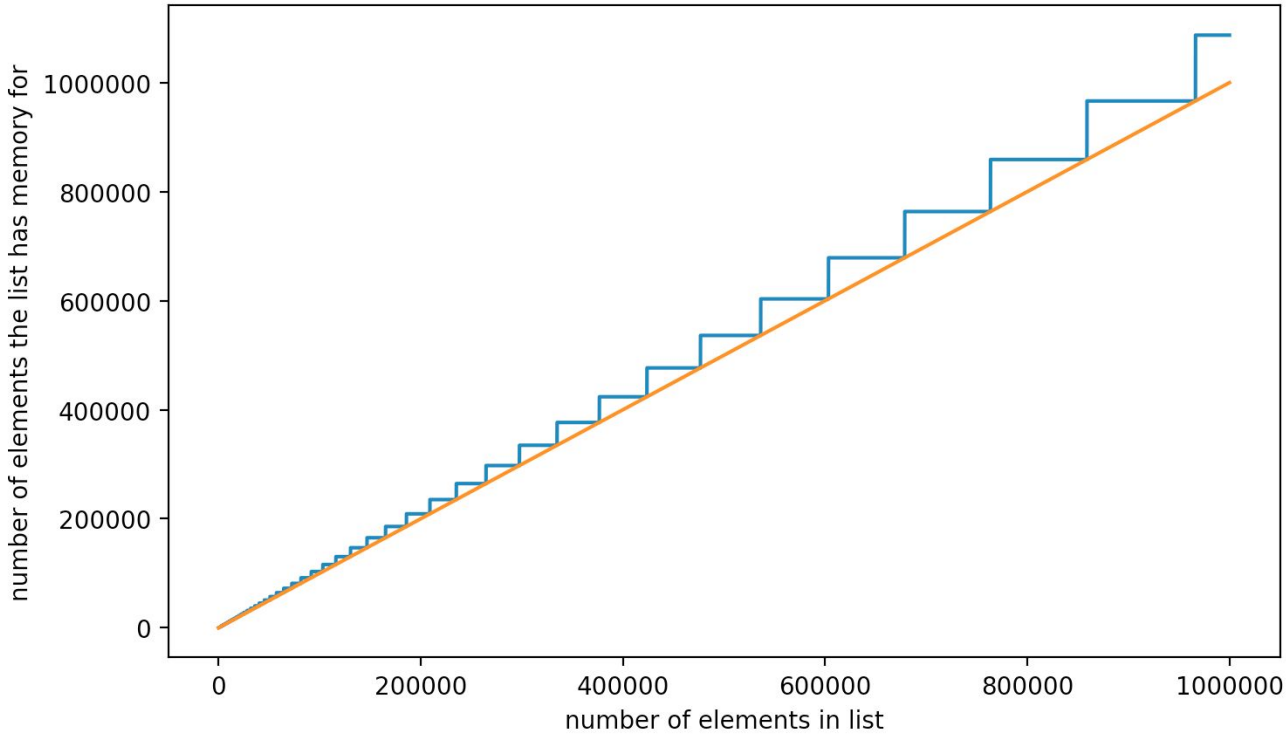
Over-allocation (Над-алокація)
Виділення пам'яті "про запас" зменшує кількість системних викликів realloc.

Amortized O(1)
Більшість операцій append() просто записують дані у вільний слот.

Wasted Space
До ~12% пам'яті може бути невикористаною в очікуванні нових даних.

Ріст Capacity vs Size
Step-wise memory allocation

Used Size · Allocated Capacity



Приблизна формула росту CPython (спрощено)

```
new_allocated = newsize + (newsize >> 3) + (newsize < 9 ? 3 : 6)
```

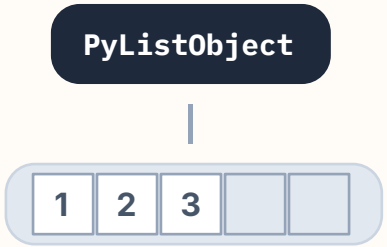
Формула обчислює нову capacity при переповненні з ростом ~12.5%

Кортежі vs Списки: пам'ять

List []

MUTABLE

FRAGMENTED ALLOCATION



Over-allocation (Resize room)

sys.getsizeof([])	56 bytes
sys.getsizeof([1,2,3])	~88 bytes

Потребує два блоки пам'яті та зберігає зайві посилання для `append()`.

Runtime Cached

Tuple ()

IMMUTABLE

COMPACT ALLOCATION

Single Block (Struct + Items)

sys.getsizeof(())	40 bytes
sys.getsizeof((1,2,3))	64 bytes

Алокується як один безперервний блок. Фіксований розмір = менше метаданих.

Структура пам'яті словників

Стара реалізація

PYTHON < 3.6

Sparse Hash Table: Велика таблиця, де кожен слот зберігає хеш, ключ і значення. Багато порожнього місця.

0	NULL
1	NULL
2	hash1 | "key" | val
3	NULL
4	NULL
5	hash2 | "id" | 42
6	NULL

High Memory Overhead

Compact Dict

PYTHON 3.6+

Dense Array + Indices: Маленький індексний масив вказує на щільний масив записів. Зберігає порядок вставки.

1. INDICES ARRAY (SPARSE, 1 BYTE / ITEM)

-1	-1	0	-1	-1	1	-1	-1

2. ENTRIES ARRAY (DENSE, NO GAPS)

0	hash1 | "key" | val
1	hash2 | "id" | 42

-25%
MEMORY USAGE

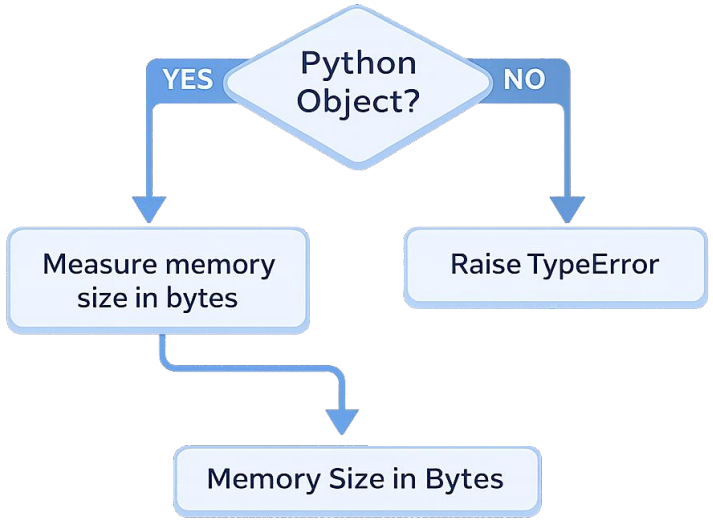
Ordered
INSERTION ORDER

Інструменти аналізу пам'яті

sys.getsizeof

BUILT-IN

для примітивів




```
sys.getsizeof(obj)
```

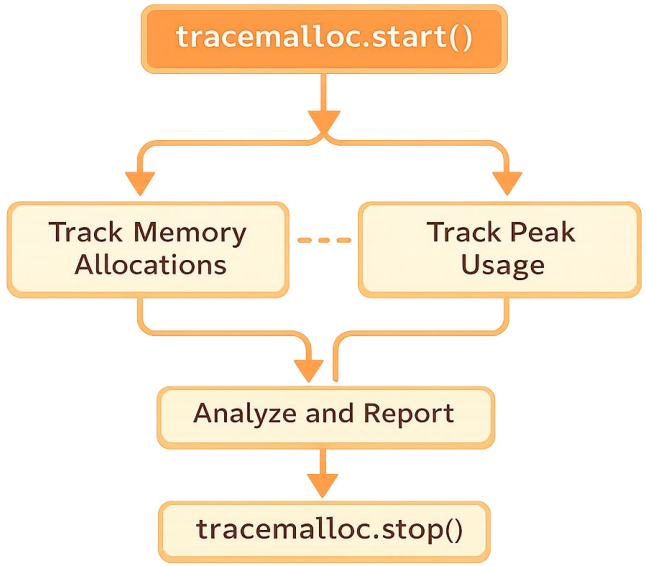
Повертає розмір самого об'єкта в байтах. Не враховує вкладені об'єкти (списки, атрибути). Для глибокого аналізу потрібна рекурсія.

✓ **Швидка перевірка простих типів**

tracemalloc

STANDARD LIB

для діагностики витоків




```
snapshot.compare_to()
```

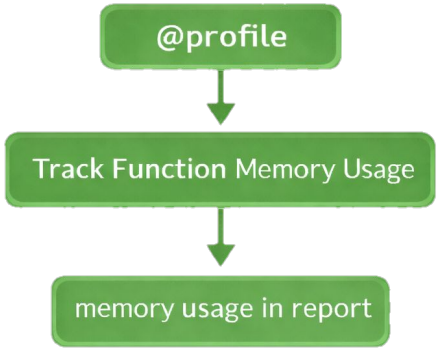
Відстежує алокації пам'яті. Дозволяє робити "знімки" (snapshots) і порівнювати їх, щоб знайти джерело витоків (Memory Leaks).

Пошук витоків пам'яті

memory_profiler

EXTERNAL (PIP)

для рефакторингу




```
@profile decorator
```

Показує споживання пам'яті рядок за рядком. Допомагає знайти конкретну ділянку коду, яка споживає найбільше RAM.

Точкова оптимізація функцій

Ключові тези: Управління пам'яттю

01

Багаторівнева алокація

CPython не просто викликає malloc. Він використовує ієрархію Arena → Pool → Block для управління малими об'єктами (< 512 байт). Це мінімізує фрагментацію та кількість системних викликів.

02

Автоматичне управління

Основний механізм - це Reference Counting (швидкий, детермінований). Garbage Collector (покоління 0-2) працює лише для того, щоб знаходити та розривати ізольовані циклічні посилання, які RC не бачить.

03

Вбудовані оптимізації

Інтернування малих чисел (-5..256) та рядків економить пам'ять. Списки та словники використовують над-алокацію (over-allocation), щоб забезпечити амортизовану складність O(1) при додаванні елементів.

04

Інструменти аналізу

Не вгадуйте. Використовуйте sys.getsizeof() для простих перевірок, tracemalloc для відстеження витоків та objgraph для візуалізації зв'язків між об'єктами.

"Пам'ять у Python - це не магія, а інженерне рішення. Розуміння вартості об'єктів дозволяє писати ефективний код, який масштабується."

FINAL THOUGHT

Що почитати?

- CPython C-API: Object Structure
- Memory Management
- Garbage Collector internals
- CPython - Garbage Collector
- CPython - obmalloc.c
- CPython - gcmodule.c
- Tracemalloc